

Manual Técnico de Infraestructura — Atena



Manual Técnico de Infraestructura en la Nube

Proyecto Atena — NeuroK AR (Fundación Universitaria Konrad Lorenz)

Documento preparado para entrega institucional al Laboratorio de Neurociencias
Aplicadas – NeuroK

Fecha: Agosto 2026

Autora: Viviana Marcela García Valderrama

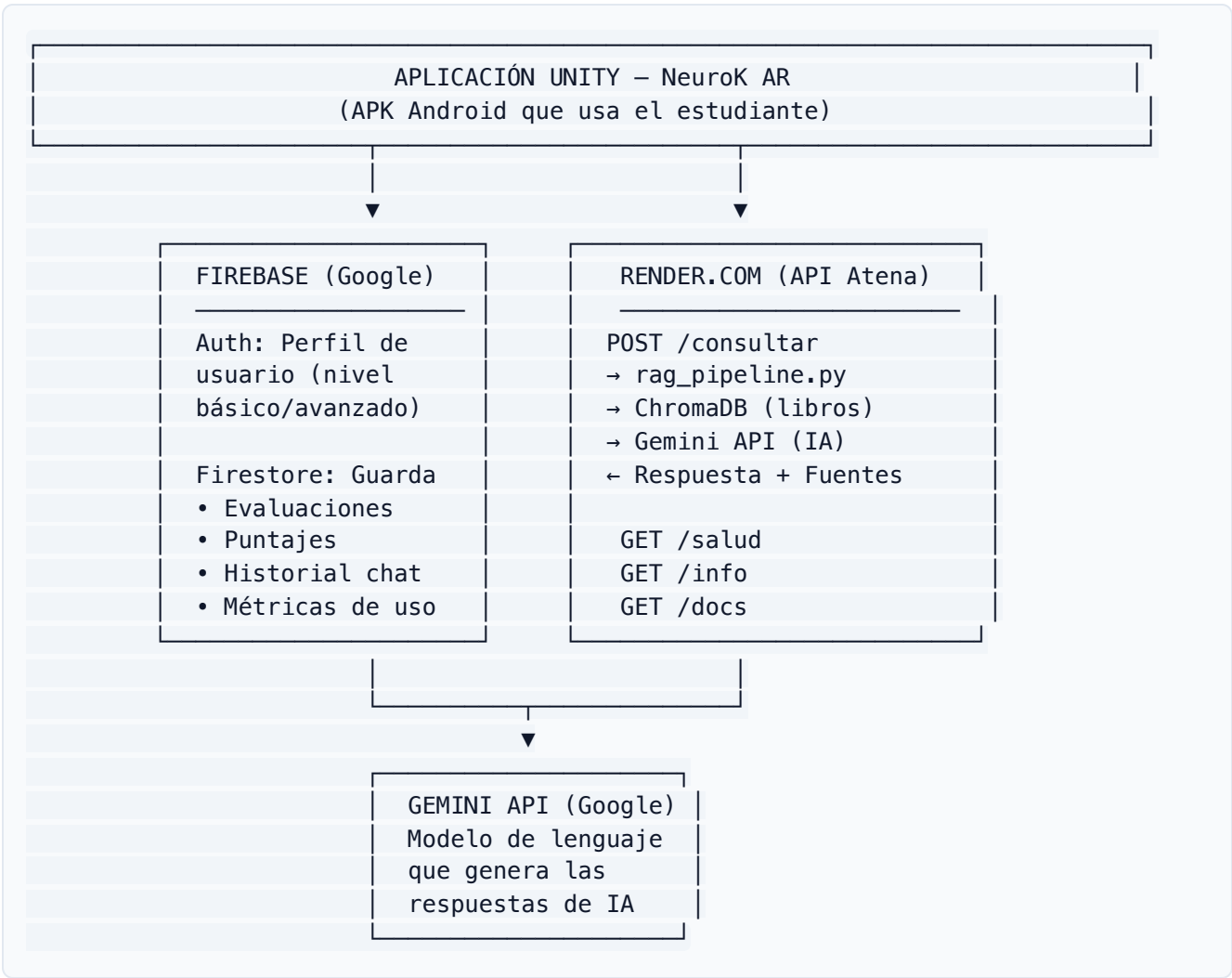


Tabla de Contenidos

1. [Resumen del Ecosistema](#)
 2. [Cuenta Institucional del Proyecto](#)
 3. [Repositorio de Código — GitHub](#)
 4. [Backend en la Nube — Render.com](#)
 5. [Base de Datos NoSQL — Firebase Firestore](#)
 6. [Modelo de IA — Gemini API \(Google AI Studio\)](#)
 7. [API REST — Endpoints y Funcionamiento](#)
 8. [Script para Unity — AtenaClient.cs](#)
 9. [Cómo Agregar y Vectorizar Nueva Literatura Científica](#)
 10. [Cómo Administrar el Sistema](#)
 11. [Cómo Actualizar el Código y el Despliegue](#)
 12. [Ficha Técnica Final de Entrega](#)
-

1. Resumen del Ecosistema

El proyecto está compuesto por **cuatro capas** que trabajan juntas de forma integrada:



2. Cuenta Institucional del Proyecto

Para garantizar la **soberanía institucional** del proyecto (que la Fundación Universitaria Konrad Lorenz sea la propietaria real de todos los servicios en la nube), se creó una cuenta oficial centralizada:

Campo	Valor
Correo Institucional	atena.unikonrad@gmail.com

Campo	Valor
Contraseña	Se entregará de forma privada al administrador
Propósito	Cuenta centralizada para administrar Render.com, Firebase Console y Google AI Studio
Vinculada a	Render.com, Firebase Console (Firestore), Google AI Studio (Gemini API)

 **Seguridad:** Esta credencial es custodiada por el Laboratorio de Neurociencias Aplicadas – NeuroK como cuenta institucional oficial del sistema.

3. Repositorio de Código — GitHub

¿Qué es GitHub?

GitHub es la plataforma donde vive el **código fuente completo del sistema**. Es el punto de verdad (Single Source of Truth) de la arquitectura. Cualquier actualización que se haga al código se sube aquí primero, y automáticamente se propaga al despliegue en Render.

Datos del Repositorio

Campo	Valor
URL del Repositorio	https://github.com/Vivi271/Atena
Tipo de Repositorio	Público (acceso abierto para compilación y auditoría)
Rama principal	<code>main</code>

Estructura del Repositorio

```
Atena/  
├─ api.py           ← Servidor FastAPI (endpoints REST)  
├─ rag_pipeline.py  ← Lógica del RAG (búsqueda + IA)
```

— config.py	← Configuración del sistema
— database.py	← Gestión de métricas locales
— app.py	← Interfaz Streamlit (local)
— AtenaClient.cs	← Script de C# para Unity
— Dockerfile	← Receta de construcción del servidor
— requirements_docker.txt	← Librerías Python necesarias
— .dockerignore	← Archivos excluidos del contenedor
— .gitignore	← Archivos ignorados por Git
— components/	← Componentes de interfaz de usuario
— Docs/	← Literatura científica indexada
— El cerebro y la conducta...pdf	
— Neuroanatomia clinica 26va...pdf	
— MODELO NEUROANATÓMICO 3D.docx	

4. Backend en la Nube — Render.com

Proceso de Despliegue Institucional Paso a Paso

El despliegue se realizó directamente con la **cuenta institucional del proyecto**:

Paso 1 — Registro con la cuenta institucional: - Se ingresó a dashboard.render.com/register - Se seleccionó **"Sign up with Google"** usando el correo **atena.unikonrad@gmail.com**

Paso 2 — Creación del Web Service: - En el panel principal, se hizo clic en **"New +"** → **"Web Service"** - Se conectó mediante el repositorio público: <https://github.com/Vivi271/Atena>

Paso 3 — Configuración del servicio: | Parámetro | Valor Configurado | |———|———| |
Name | Atena | | **Region** | Oregon (US West) | | **Runtime** | Docker (detectado automáticamente desde el repositorio) | | **Branch** | main | | **Instance Type** | Free (\$0 USD/mes permanente) | |
Auto-Deploy | On Commit (actualizaciones continuas al hacer cambios en GitHub) | | **Health Check Path** | /salud |

Paso 4 — Variable de Entorno: | Clave | Valor | |———|———| | **GEMINI_API_KEY** | (Clave API de Google Gemini generada para el proyecto) |

Paso 5 — Lanzamiento y Estado: - Se ejecutó **"Deploy Web Service"** - Estado:  **Live** (Desplegado y en producción)

URL Oficial de Producción

<https://atena-vugz.onrender.com>

5. Base de Datos NoSQL — Firebase Firestore

Proceso de Configuración Institucional

Paso 1 — Creación del Proyecto: - Se ingresó a console.firebase.google.com con `atena.unikonrad@gmail.com` - Se creó el proyecto **Atena** (ID asignado: `atena-2d765`)

Paso 2 — Habilitación de Cloud Firestore: - En el menú lateral se seleccionó **Bases de datos y almacenamiento** → **Cloud Firestore** - **Edición:** Standard - **Ubicación de Servidores:** `nam5` (United States) - **Reglas de Seguridad:** Modo de prueba (lectura y escritura desde Unity) - Se habilitó la base de datos con éxito

6. Modelo de IA — Gemini API (Google AI Studio)

Función	Modelo
Generación de Respuestas RAG	<code>gemini-2.5-flash</code>
Vectorización Semántica	<code>gemini-embedding-001</code>

La gestión de cuotas y claves se realiza directamente desde aistudio.google.com con la cuenta `atena.unikonrad@gmail.com` .

7. API REST — Endpoints y Funcionamiento

URL Base de Producción

`https://atena-vugz.onrender.com`

Catálogo de Endpoints

1. GET /docs — Documentación Interactiva (Swagger UI)

Interfaz gráfica para interactuar con la API directamente desde el navegador web. - **Enlace:** <https://atena-vugz.onrender.com/docs>

2. GET /salud — Health Check

Verifica que el servicio esté activo y que el almacén vectorial esté cargado. - **Enlace:** <https://atena-vugz.onrender.com/salud>

3. GET /info — Ficha Técnica del Servicio

Muestra metadatos y modelos configurados en el pipeline. - **Enlace:** <https://atena-vugz.onrender.com/info>

4. POST /consultar — Inferencia RAG (Consumo desde Unity)

Endpoint que procesa las preguntas neuroanatómicas.

8. Script para Unity — AtenaClient.cs

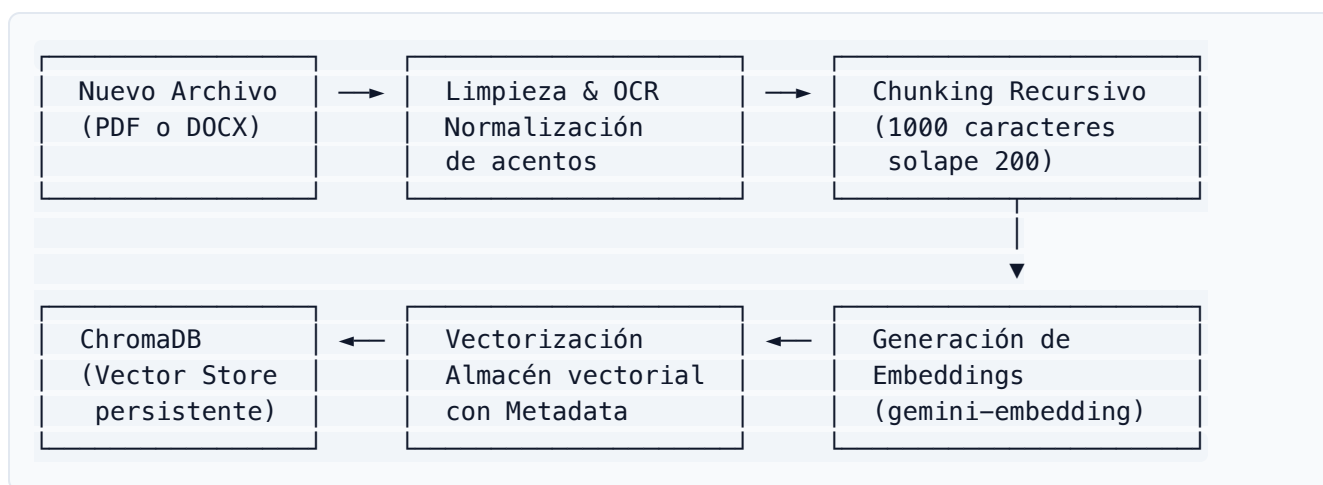
Script oficial en C# para la aplicación móvil NeuroK AR (Unity / Android): - **Archivo:** [AtenaClient.cs](#) - **URL Base integrada:** <https://atena-vugz.onrender.com>

9. Cómo Agregar y Vectorizar Nueva Literatura Científica

El sistema está diseñado para ser **extensible y modular**. Si los docentes o investigadores del Laboratorio de NeuroK desean incorporar nuevos libros, guías clínicas o artículos científicos en el futuro, el proceso de vectorización es automático.

¿Cómo funciona el Pipeline de Vectorización?

Cuando se agrega un documento, el motor RAG (`rag_pipeline.py`) ejecuta automáticamente las siguientes etapas:



Método 1: Actualización Automática en la Nube (Vía GitHub — Recomendado)

Es el método más sencillo y no requiere instalar nada en la computadora:

1. Subir el nuevo archivo:

- Entrar al repositorio: <https://github.com/Vivi271/Atena/tree/main/Docs>
- Hacer clic en el botón superior **"Add file"** ➡ **"Upload files"**.
- Arrastrar el nuevo archivo PDF o DOCX dentro de la carpeta `Docs/`.
- Escribir un mensaje de commit (ej. `docs: agregar Guia_Neuroanatomia_2026.pdf`) y hacer clic en **"Commit changes"**.

2. Re-vectorización Automática:

- Render detecta el nuevo commit en GitHub de forma automática (*Auto-Deploy*).
- El contenedor lee la carpeta `Docs/`, procesa el nuevo archivo, genera los embeddings vectoriales con Gemini y actualiza la base de datos ChromaDB en la nube.
- En ~2 minutos, las nuevas consultas en Unity ya tendrán en cuenta la nueva literatura.

Método 2: Indexación Local desde el Panel Web (Streamlit UI)

Si se desea probar la vectorización de forma interactiva en la computadora antes de subirla a la nube:

1. Iniciar la aplicación web local:

```
streamlit run app.py
```

2. En la barra lateral izquierda, ingresar al **"Panel de Administración"** con el PIN de acceso:

PIN: 1234

3. En la sección **"Gestión de Documentos"**:

- Arrastrar el nuevo archivo PDF o DOCX en el cargador de archivos.
- Hacer clic en **"Indexar Nuevos Documentos"** (indexación incremental) o **"Reconstruir VectorDB"** (re-indexación completa desde cero).

4. El sistema mostrará una barra de progreso indicando cuántos fragmentos (*chunks*) fueron creados y vectorizados exitosamente.

10. Cómo Administrar el Sistema

La administración está completamente centralizada en la cuenta institucional:

Plataforma	URL de Administración	Cuenta de Acceso
Render.com (Servidor API)	dashboard.render.com	<code>atena.unikonrad@gmail.com</code>
Firebase Console (Base de Datos)	console.firebase.google.com	<code>atena.unikonrad@gmail.com</code>
Google AI Studio (API Key Gemini)	aistudio.google.com	<code>atena.unikonrad@gmail.com</code>
GitHub (Código Fuente)	github.com/Vivi271/Atena	Repositorio público

11. Cómo Actualizar el Código y el Despliegue

Despertar el Servidor en Demostraciones

La capa gratuita de Render entra en modo de reposo tras 15 minutos sin peticiones. Para una sustentación o demostración en vivo: - Abrir la URL `https://atena-vugz.onrender.com/salud` en el navegador 1 minuto antes para que el servidor responda de inmediato en Unity.

12. Ficha Técnica Final de Entrega

Parámetro	Detalle Institucional
Nombre del Proyecto	Atena — Consultor RAG en Neuroanatomía
Aplicación Móvil Cliente	NeuroK AR (Unity / Android)
Institución	Fundación Universitaria Konrad Lorenz
Laboratorio	Neurociencias Aplicadas – NeuroK
Autora	Viviana Marcela García Valderrama
Año	2026
URL Base de Producción	https://atena-vugz.onrender.com
Documentación Interactiva	https://atena-vugz.onrender.com/docs
Health Check	https://atena-vugz.onrender.com/salud
Repositorio Oficial	https://github.com/Vivi271/Atena
Cuenta Institucional Delegada	atena.unikonrad@gmail.com
Base de Datos NoSQL	Firebase Cloud Firestore (atena-2d765)
Motor de IA	Google Gemini 2.5 Flash + Gemini Embedding 001
Infraestructura Cloud	Render.com (Docker Container)
Costo Operativo Mensual	\$0 USD

Documento técnico de entrega oficial — Proyecto de Grado — Facultad de Psicología — Fundación Universitaria Konrad Lorenz.